

Discrete Ordinates Method

Part One: Uncollided Radiation in a Leaf Canopy

A Companion Guide to dom_uncollided.py

Physical Models in Remote Sensing — Chapter 04, Part 03

How To Use This Document

This document and the Python file `dom_uncollided.py` are designed to be used together. Every section in this document corresponds directly to a numbered section in the code. When you read about a concept here, you can find exactly where it is implemented in the code by looking for the matching section header.

This document explains:

- The physical problem being solved
- Why each step is necessary
- The mathematical equations
- Simple diagrams of the algorithm
- What each function in the code does

The code provides:

- A working implementation you can run
- Heavily commented functions
- Meaningful variable names
- A clearly marked input section
- Output plots for verification

Where to Start

1. Read Sections 1 and 2 of this document to understand the problem and geometry.
2. Open `dom_uncollided.py` and go to Section 11 (the bottom). This is where you set inputs.
3. Run the code: `python dom_uncollided.py`
4. Come back and read each section as you work through the code.

1. The Problem We Are Solving

1.1 What Is a Leaf Canopy?

A leaf canopy is a layer of vegetation through which sunlight must pass to reach the ground. Think of a forest or a field of crops. Sunlight enters from above, bounces off and through leaves many times, and some of it eventually escapes back upward. Remote sensing instruments measure this upward-escaping light to infer properties of the vegetation.

1.2 The Two-Part Strategy

Modelling radiation in a canopy is complicated because photons scatter off leaves repeatedly. We split the total radiation field I into two parts:

```
Total radiation field I = I0 + IC

I0 = UNCOLLIDED radiation
    Photons that have NOT yet hit any leaf since entering the canopy.
    These travel in straight lines -> Beer-Lambert attenuation only.
    Solved analytically in THIS CODE.

IC = COLLIDED radiation
    Photons that have hit at least one leaf and been scattered.
    These require a more complex iterative solver.
    Solved in PART TWO (next code file).
```

Figure 1: Splitting the radiation field into uncollided and collided parts

The uncollided field I_0 is computed first because it is mathematically simple (pure exponential decay). It then provides a source term Q , called the First Collision Source, which drives the collided solver in Part Two.

1.3 The Governing Equation for Uncollided Photons

The general 1D radiative transfer equation is:

$$-\mu * dI/dL + G(\Omega) * I(L, \Omega) = \text{scattering source}$$

For UNCOLLIDED photons the scattering source is zero by definition (these photons have not scattered yet). So the equation simplifies to:

$$-\mu * dI_0/dL + G * I_0(L, \Omega) = 0$$

This is a simple first-order ordinary differential equation. Its solution is the Beer-Lambert law:

$$I_0(L_2, \Omega) = I_0(L_1, \Omega) * \exp[-G * |L_2 - L_1| / |\mu|]$$

where: G = extinction cross section (= 0.5 for uniform leaves)
mu = cos(theta), theta = direction angle from zenith
L = cumulative leaf area index (depth into canopy)

Figure 2: The Beer-Lambert solution for uncollided radiation

2. Coordinate System and Sign Convention

2.1 The Spatial Variable L

Instead of geometric depth z (in metres), we use the cumulative Leaf Area Index L :

```

Zenith (-z axis, pointing UP)
|
L = 0 +----- canopy TOP    (where radiation enters)
|
|   leaf canopy
|   (leaves intercept photons as they travel through)
|
L=LAI +----- canopy BOTTOM = ground surface
|
///   ground (reflects radiation upward)

```

Figure 3: The spatial coordinate L (cumulative leaf area index)

L increases downward. $L = 0$ at the top, $L = LAI$ at the bottom. Using L instead of z is natural because what matters for photon interception is how much leaf area the photon has passed through, not how many metres it has travelled.

2.2 The Directional Variable Omega

A photon's direction Omega = (theta, phi) is described by two angles:

Polar angle theta:

Measured from the UPWARD zenith (-z axis).

$$\mu = \cos(\theta)$$

$\mu > 0$ means UPWARD ($\theta < 90$ deg)

$\mu < 0$ means DOWNWARD ($\theta > 90$ deg)

Azimuthal angle phi:

Measured in the horizontal plane.

$\phi = 0$ deg is toward the sun.

$\phi = 180$ deg is away from the sun.

```

UPWARD hemisphere (mu > 0, theta < 90 deg)
^
|  mu > 0
|
-----+----- horizontal plane
|
|  mu < 0
|
v
DOWNWARD hemisphere (mu < 0, theta > 90 deg)

The SUN enters from the downward hemisphere: mu_solar < 0
Example: theta_solar = 140 deg -> mu_solar = cos(140 deg) = -0.766

```

Figure 4: The sign convention for μ (matches lecture slides)

3. Algorithm Overview

The uncollided DOM solver proceeds in six steps, corresponding to Steps 1-6 in the lecture slides. The diagram below shows the flow:

```

INPUT: LAI, f_dir, theta_solar, phi_solar, rho_g, rho_L, tau_L
      N (polar directions), M (azimuthal directions), K (layers)
      |
      v
STEP 1: Angular discretization
      Compute N Gauss-Legendre points in mu and M uniform points in phi.
      Result: polar_cosines[1..N], azimuthal_angles[1..M], weights
      |
      v
STEP 2: Spatial discretization
      Divide canopy into K layers of thickness delta_L = LAI/K.
      Result: cell_edge_LAI_values[k=1..K+1]
      |
      v
STEP 3a: Downward sweep (direct solar beam)
      Apply Beer-Lambert decay from k=1 to k=K+1 for the solar direction.
      Result: direct_solar_intensity_at_cell_edges[k=1..K+1]
      |
STEP 3b: Downward sweep (diffuse sky)
      Apply Beer-Lambert decay for all N/2 downward quad. directions.
      Result: uncollided_intensity_at_cell_edges[i=1..N/2, j, k=1..K+1]
      |
      v
STEP 4: Lower boundary condition
      Lambertian ground reflects all downward flux upward.
      Result: ground_reflected_intensity_at_bottom[i=N/2+1..N, j]
      |
      v
STEP 5: Upward sweep
      Apply Beer-Lambert decay upward for all N/2 upward directions.
      Result: uncollided_intensity_at_cell_edges[i=N/2+1..N, j, k=1..K+1]
      |
      v
STEP 6: First Collision Source  Q = Q1 + Q2 + Q3
      Compute the source for collided photons at each cell centre.
      Result: first_collision_source[i=1..N, j, k=1..K]
      |
      v
OUTPUT: Fluxes, BRF plots, FCS arrays  (ready for Part Two solver)

```

Figure 5: Algorithm flowchart for the uncollided DOM solver

4. Step 1: Angular Discretization (Code Section 1)

4.1 Why Discretize Angles?

The radiative transfer equation involves integrals over all directions on the sphere. For example, the downward flux at depth L requires integrating intensity over the entire downward hemisphere. We cannot do this analytically in general, so we replace the integral with a finite weighted sum at N chosen angles:

```
EXACT:      integral_{-1}^{+1} f(mu) dmu

APPROX:      sum_{i=1}^{N} w_i * f(mu_i)

The quadrature CHOOSES mu_i and w_i to make the approximation as
accurate as possible for a given N.
```

Figure 6: Replacing a continuous integral with a quadrature sum

4.2 Gauss-Legendre Quadrature

Gauss-Legendre quadrature places the N sample points (ordinates) at locations that are optimal for polynomial integration. With N points it integrates any polynomial up to degree $2N-1$ exactly. For smooth functions it is far more efficient than equally-spaced sampling.

numpy computes the ordinates and weights automatically:

```
gauss_ordinates, gauss_weights = np.polynomial.legendre.leggauss(N)

# Result for N=4:
# ordinates ~ [-0.861, -0.340, +0.340, +0.861] (sorted ascending)
# weights   ~ [ 0.348,  0.652,  0.652,  0.348]
```

4.3 Index Convention

We keep numpy's natural ascending order so that:

```
polar_cosines[i]    for i = 1 .. N/2      --> mu_i < 0  (DOWNWARD)
polar_cosines[i]    for i = N/2+1 .. N    --> mu_i > 0  (UPWARD)

Arrays are padded with a dummy index-0 slot (always zero, never used),
so that array index i directly matches the slide notation i = 1..N.

Example with N=4:
polar_cosines[0] = 0.000 (dummy, unused)
polar_cosines[1] = -0.861 (downward, mu < 0)
polar_cosines[2] = -0.340 (downward, mu < 0)
polar_cosines[3] = +0.340 (upward,   mu > 0)
polar_cosines[4] = +0.861 (upward,   mu > 0)
```

Figure 7: How polar cosines are stored in the array

4.4 Sanity Check (Slide 9)

Two checks verify the quadrature is correct:

```
Check 1:  sum of all w_i          = 2.0    (integral of 1 over [-1,+1])
Check 2:  sum of mu_i*w_i (i>N/2) = 0.5    (integral of mu over [0,+1])
```


5. Step 2: Spatial Discretization (Code Section 1)

We divide the canopy into K layers of equal thickness in LAI units:

```

cell edge k=1      L = 0                canopy TOP
-----
      Layer 1      thickness = delta_L = LAI/K
-----
cell edge k=2      L = 1 * delta_L
-----
      Layer 2
-----
cell edge k=3      L = 2 * delta_L
      ...
cell edge k=K      L = (K-1) * delta_L
-----
      Layer K
-----
cell edge k=K+1    L = LAI              canopy BOTTOM = ground
////////// ground //////////////////////////////////////

Cell centre k+0.5 is the MIDPOINT of layer k (between edges k and k+1).
The First Collision Source Q is evaluated at cell centres.
```

Figure 8: The spatial grid with K layers and K+1 cell edges

Key rule from the slides

Cell edges go from k=1 (canopy top) to k=K+1 (canopy bottom).

Cell centres are at k=1.5, 2.5, ..., K+0.5 (K centres total).

In code we write k+(1/2) as just k, indexing from 1 to K.

6. Steps 3a and 3b: Downward Sweeps (Code Section 4)

6.1 Direct Solar Beam (Step 3a, Slide 12)

The solar beam is a single collimated direction $\Omega_{\text{solar}} = (\mu_{\text{solar}}, \phi_{\text{solar}})$. It enters the canopy top at $k=1$ and decays layer by layer:

```
k=1  I0_dir[1] = I_solar          <-- set by upper boundary condition
k=2  I0_dir[2] = I0_dir[1] * exp[-G * delta_L / |mu_solar|]
k=3  I0_dir[3] = I0_dir[2] * exp[-G * delta_L / |mu_solar|]
...
k=K+1 I0_dir[K+1] = I0_dir[K] * exp[-G * delta_L / |mu_solar|]

The attenuation factor exp[-G * delta_L / |mu_solar|] is the same
for every layer (G and delta_L are constant throughout the canopy).
So we compute it ONCE and multiply repeatedly.
```

Figure 9: Sweeping the direct solar beam downward (Beer-Lambert)

6.2 Diffuse Sky Radiation (Step 3b, Slide 13)

Each of the $N/2$ downward quadrature directions sweeps independently. A near-horizontal direction (small $|\mu_i|$) has a larger path-length factor $1/|\mu_i|$ and attenuates faster:

```
For each downward direction i = 1..N/2  (mu_i < 0):

    one_layer_attenuation = exp[-G * delta_L / |mu_i|]

    k=1:  I0_dif[i,j,1] = Id          <- boundary condition
    k=2:  I0_dif[i,j,2] = I0_dif[i,j,1] * one_layer_attenuation
    k=3:  I0_dif[i,j,3] = I0_dif[i,j,2] * one_layer_attenuation
    ...   (continue to k=K+1)

Do this for ALL (N/2) * M downward directions.
```

Figure 10: Sweeping diffuse sky radiation downward for each direction

7. Step 4: Lower Boundary Condition (Code Section 3)

7.1 Lambertian Ground Reflection

When downward uncollided radiation reaches the ground ($k=K+1$), the ground reflects it. We model the ground as a Lambertian reflector: it scatters all incident radiation equally in ALL upward directions with reflectance ρ_g .

```
At the ground (k=K+1):

Total downward flux F_down = |mu_solar| * I0_dir[K+1]          <- direct beam
                             + sum_{n=1}^{N/2} sum_{m=1}^{M}
                               w_n * w_phi * |mu_n| * I0_dif[n,m,K+1]
                                                           <- diffuse sky

Reflected intensity (the SAME in all upward directions):

I_reflected = (rho_g / pi) * F_down

Set I0_dif[i, j, K+1] = I_reflected for ALL upward directions i=N/2+1..N
```

Figure 11: Lambertian ground reflection boundary condition (Slide 14)

Why is the reflected intensity the same in all directions?

A Lambertian surface reflects equally bright in every direction.

This is the definition of an ideal diffuse (matte) reflector.

Soil and most natural ground surfaces are approximately Lambertian.

8. Step 5: Upward Sweep (Code Section 4)

After the ground reflection sets the intensity at $k=K+1$ for all upward directions, we sweep upward through the canopy layers. The same Beer-Lambert formula applies, but now we go from $k=K+1$ back up to $k=1$:

```
For each upward direction  $i = N/2+1..N$  ( $\mu_i > 0$ ):  
  
    one_layer_attenuation =  $\exp[-G * \text{delta\_L} / |\mu_i|]$   
  
     $k=K+1$ :  $I0\_dif[i,j,K+1] = I\_reflected$     <- ground BC  
     $k=K$ :    $I0\_dif[i,j,K] = I0\_dif[i,j,K+1] * one\_layer\_attenuation$   
     $k=K-1$ :  $I0\_dif[i,j,K-1] = I0\_dif[i,j,K] * one\_layer\_attenuation$   
    ...    (continue up to  $k=1$ )  
  
Do this for ALL  $(N/2) * M$  upward directions.  
  
After this step we have uncollided intensities at EVERY cell edge  
for EVERY direction (downward sky + upward ground-reflected).
```

Figure 12: Sweeping ground-reflected radiation upward (Slide 16)

9. Step 6: First Collision Source (Code Section 6)

9.1 What Is It?

The First Collision Source $Q(L_{\{k+0.5\}}, \Omega_{\{ij\}})$ is the rate at which uncollided photons are scattered INTO direction $\Omega_{\{ij\}}$ at cell centre $k+0.5$ per unit volume. It is the source term that drives the collided radiation equation in Part Two.

Q has three contributions, one from each source of uncollided radiation:

9.2 Q1: From the Direct Solar Beam (Slide 18)

The solar beam scatters into all directions. Since the beam is a single direction with no quadrature sum needed:

```
Q1[i, j, k] = (1/pi) * Gamma(Omega_solar -> Omega_{ij})
               * I0_dir_centre[k]

where I0_dir_centre[k] = 0.5 * (I0_dir[k] + I0_dir[k+1]) <- cell centre

Evaluated for ALL output directions i=1..N (both up and down).
```

Figure 13: First Collision Source Q1 from the direct solar beam

9.3 Q2: From Downward Diffuse Sky Photons (Slide 19)

Downward diffuse sky photons that scatter for the very first time. The source sum runs over DOWNWARD directions only ($n=1..N/2$, $\mu_n < 0$). The output is evaluated for ALL directions (Slide 19: Ω_{ij} represents all directions in a sphere):

```
Q2[i, j, k] = (1/pi) * sum_{n=1}^{N/2} sum_{m=1}^M
               w_n * w_phi * Gamma(Omega_{nm} -> Omega_{ij})
               * I0_dif_centre[n, m, k]

Source: n = 1..N/2 (DOWNWARD directions, mu_n < 0).
Output: i = 1..N (ALL directions, both up and down).
Slide 19 note: 'The integration done only for downward directions'
means the SOURCE SUM is restricted to n=1..N/2, not the output.
```

Figure 14: First Collision Source Q2 from downward diffuse sky (Slide 19)

9.4 Q3: From Upward Ground-Reflected Photons (Slide 20)

Upward ground-reflected photons that scatter for the very first time. The source sum runs over UPWARD directions only ($n=N/2+1..N$, $\mu_n > 0$). The output is evaluated for ALL directions (Slide 20: Ω_{ij} represents all directions in a sphere):

```
Q3[i, j, k] = (1/pi) * sum_{n=N/2+1}^{N} sum_{m=1}^{M}
               w_n * w_phi * Gamma(Omega_{nm} -> Omega_{ij})
               * I0_dif_centre[n, m, k]

Source: n = N/2+1..N   (UPWARD directions, mu_n > 0).
Output: i = 1..N       (ALL directions, both up and down).
Slide 20 note: 'The integration done for upward directions'
means the SOURCE SUM is restricted to n=N/2+1..N, not the output.
```

Figure 15: First Collision Source Q3 from upward ground-reflected (Slide 20)

9.5 Total First Collision Source

```
Q[i, j, k] = Q1[i, j, k] + Q2[i, j, k] + Q3[i, j, k]

This Q array is the OUTPUT of this code and the INPUT to Part Two.
```

Figure 16: Total First Collision Source

10. The Volume Scattering Phase Function Gamma (Code Section 2)

10.1 Physical Meaning

$\text{Gamma}(\Omega' \rightarrow \Omega) * d\Omega$ is the probability that a photon arriving from direction Ω' is scattered into the solid angle $d\Omega$ around Ω after hitting a leaf. It encodes how much a leaf reflects and transmits light in each direction.

10.2 Formula for Uniform Leaf Normals

For a uniform (spherical) distribution of leaf normal orientations, the phase function has an exact analytical expression (from lecture C4-P1):

```
Gamma(Ω' -> Ω) = (omega_L / (3*pi)) * (sin(beta) - beta*cos(beta))
                  + (tau_L / 3.0)      * cos(beta)
```

where:

```
beta      = scattering angle = arccos(Ω' . Ω)
omega_L   = leaf albedo = rho_L + tau_L
tau_L     = leaf transmittance
```

The dot product of two directions in spherical coordinates:

```
Ω' . Ω = mu1*mu2 + sqrt(1-mu1^2)*sqrt(1-mu2^2)*cos(phi1-phi2)
```

Figure 17: The phase function formula for uniform leaf normals

10.3 Why Pre-compute?

We need Gamma for every pair of directions: $(\Omega_{nm} \rightarrow \Omega_{ij})$. For $N=16$ and $M=16$ that is $16*16*16*16 = 65,536$ evaluations. Since Gamma does not depend on depth L or time, we compute it ONCE before the main calculation and store it in the array `gamma_quad_to_quad`. This avoids recomputing the same value $K*N*M$ times inside the loops.

11. Outputs and Plots

11.1 Fluxes

After computing uncollided intensities at all cell edges and directions, we integrate over each hemisphere to get fluxes:

```

Downward direct flux:   F_dir_down[k] = |mu_solar| * I0_dir[k]

Downward diffuse flux:  F_dif_down[k] = sum_{i=1}^{N/2} sum_{j=1}^{M}
                                w_i * w_phi * |mu_i| * I0_dif[i,j,k]

Upward flux:           F_up[k]          = sum_{i=N/2+1}^{N} sum_{j=1}^{M}
                                w_i * w_phi * |mu_i| * I0_dif[i,j,k]

All fluxes are normalised by F_in = 1 W/m2 for the plots.

```

Figure 18: Flux integration formulas

11.2 BRF

The Bidirectional Reflectance Factor at the canopy top is:

```

BRF(Omega_view) = I_up(L=0, Omega_view) / (F_in / pi)

Plotted in the PRINCIPAL PLANE (containing the sun):
Negative view angles <-- backscatter side (phi_view = phi_solar + 180 deg)
Positive view angles --> forward scatter  (phi_view = phi_solar)

```

Figure 19: BRF definition and principal plane convention

11.3 Three Output Plots

Plot	What it shows	What to expect
Plot A	Downward uncollided flux vs normalised depth L/LAI	Both direct and diffuse decrease from L/LAI=0 to 1. Direct starts at f_dir=0.7, diffuse at 0.3.
Plot B	Upward uncollided flux vs normalised depth L/LAI	Upward flux peaks at L/LAI=1 (ground level) and decreases upward as it is attenuated.
Plot C	Uncollided BRF at canopy top (L=0) in the principal plane	Symmetric bell shape peaking near nadir. NIR higher than RED due to higher leaf and ground albedo.

12. Input Parameter Reference

All inputs are set in Section 11 of `dom_uncollided.py`. Here is a complete reference with recommended values and physical interpretation:

Parameter	Typical value	Units	Physical meaning
N	16	—	Polar quadrature order. Must be even. Higher = more accurate but slower.
M	16	—	Azimuthal directions. Higher = better azimuthal resolution.
K	50	—	Number of canopy layers. Higher = finer spatial resolution.
LAI	1.5 or 4.0	m ² /m ²	Total leaf area index. 1.5 = sparse; 4.0 = dense forest.
f_dir	0.7	—	Fraction 0..1 of incident flux that is direct solar. 1 = full sun; 0 = overcast.
theta_o_deg	140.0	degrees	Solar zenith from upward zenith. Must be > 90 for a downward sun.
phi_o_deg	0.0	degrees	Solar azimuth. Sets the reference direction for the principal plane.
rho_g	0.10 (RED)	—	Ground reflectance. Typical soil: 0.10 RED, 0.20 NIR.
F_in	1.0	W/m ²	Total incident flux. Set to 1.0 so all outputs are automatically normalised.
rho_L	0.06 (RED)	—	Leaf reflectance. Leaves absorb most RED; reflect most NIR.
tau_L	0.04 (RED)	—	Leaf transmittance. Small in RED; large in NIR (leaves are nearly transparent).

13. Code Structure Map

Use this map to find any concept in the code quickly:

Section	Function name(s)	What it does
1	<code>setup_angular_quadrature_grid</code>	Gauss-Legendre quadrature, angle arrays
2	<code>compute_extinction_cross_section</code> <code>compute_volume_scattering_phase_function</code> <code>precompute_gamma_solar_to_all_quad_directions</code> <code>precompute_gamma_quad_to_quad</code>	G=0.5 and full Gamma table
3	<code>compute_upper_boundary_direct_solar_intensity</code> <code>compute_upper_boundary_diffuse_sky_intensity</code>	I_solar and I_diffuse at L=0
4	<code>sweep_direct_solar_beam_downward</code> <code>sweep_diffuse_sky_radiation_downward</code> <code>apply_lambertian_ground_boundary_condition</code> <code>sweep_ground_reflected_radiation_upward</code>	Beer-Lambert sweeps Steps 3a, 3b, 4, 5
5	<code>compute_cell_centre_intensity_1d</code> <code>compute_cell_centre_intensity_3d</code>	Edge to centre averaging
6	<code>compute_first_collision_source</code>	$Q = Q1 + Q2 + Q3$ (Steps 6a, 6b, 6c)
7	<code>integrate_fluxes_over_hemisphere</code>	F_down and F_up at all cell edges
8	<code>compute_BRF_at_canopy_top_principal_plane</code>	BRF for Plot C
9	<code>run_uncollided_dom_solver</code>	Master function, calls all steps in order
10	<code>make_three_panel_plots</code>	Generates Plots A, B, C
11	(main block)	All inputs -- START HERE to change parameters